DuocUC® INFORMÁTICA Y TELECOMUNICACIONES

Spring Security y Manejo de Logs

DESARROLLO FULLSTACK I

DSY1103

FORMAMOS PERSONAS PARA UNA SOCIEDAD MEJOR

• Contenidos

- Spring Security / JWT
- Logs estructurados con SLF4J + Logback
- Manejo de excepciones globales con @ControllerAdvice
- Desarrollo de actividades practicas.

01

Spring Security y JWT

• Seguridad Básica: JWT y Spring Security

Spring Security es el framework de seguridad dominante. Proporciona servicios de autenticación (¿quién eres?) y autorización (¿qué puedes hacer?).

Spring Security agrega:

- Filtros que se ejecutan antes de llegar al Controller
- Manejo de usuarios y roles
- Autenticación (login) y autorización (qué puede ver/hacer)

En Spring Boot 3, se configura usualmente con una clase **SecurityFilterChain**.

```java
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/**").permitAll()
            .anyRequest().authenticated()
        )
        .httpBasic(Customizer.withDefaults()); // o JWT más adelante

    return http.build();
}
```

• Seguridad Básica: JWT y Spring Security

JWT (JSON Web Token) es el estándar para la seguridad en microservicios. Es un token codificado que contiene información de usuario y permisos, permitiendo la autenticación sin estado (Stateless), ideal para la escalabilidad horizontal.

Características de JWT

1. **Autocontenido**: Contiene toda la información de usuario y roles necesaria (claims) para la autenticación sin consultar la base de datos en cada petición.
2. **Firmado (Signed)**: Utiliza una clave secreta (Secret Key) para generar una firma que garantiza la integridad del token (no ha sido alterado).
3. **Transmisión**: Se envía típicamente en el encabezado Authorization con el prefijo Bearer

• Seguridad Básica: JWT y Spring Security

Flujo básico:

1. Cliente se autentica → /auth/login
2. El servidor valida credenciales y genera un JWT firmado.
3. El cliente guarda el JWT (por ejemplo, en localStorage o memoria).
4. En cada request posterior incluye el token en el header.
5. El filtro de Spring Security valida el token antes de llegar al Controller.

Estructura del JWT

Un JWT se compone de tres partes (Header, Payload, Signature) codificadas en Base64url y separadas por puntos (.):

HEADER.PAYLOAD.SIGNATURE

- **Payload (Carga Útil):** Contiene los claims, como el ID de usuario (sub), la fecha de expiración (exp) y los roles (authorities).

• Seguridad Básica: JWT y Spring Security

Dependencias Requeridas (pom.xml)

Para implementar JWT con Spring Security, se necesitan las siguientes dependencias en el archivo pom.xml:

Spring Security Core: El starter base que provee la infraestructura de seguridad.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

JWT (JSON Web Token): La librería más utilizada para la creación, firma y validación de los tokens. Es esencial para manejar el ciclo de vida del JWT

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version> <!-- Usar la versión más reciente y estable -->
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
```

• El Cambio de Paradigma: De Stateful (Sesiones) a Stateless (JWT)

El login integrado de **Spring Security** se basa en el concepto **Stateful** (con estado), donde el servidor crea y mantiene un registro de la sesión **HTTP del usuario**. Esto crea acoplamiento y dificulta la escalabilidad horizontal.

La migración a **JWT** impone un modelo **Stateless** (sin estado): el servidor no almacena ninguna información de sesión, sino que toda la identidad del usuario reside en el token firmado.

1. **Deshabilitar Sesiones**: *El Pilar de la Arquitectura Stateless*: La configuración de Spring Security debe forzar la política de creación de sesiones a **sessionCreationPolicy(SessionCreationPolicy.STATELESS)**. Esto instruye a Spring a ignorar y no crear sesiones HTTP para las peticiones, desvinculando la autenticación del estado del servidor. Es el primer y más crítico paso para una arquitectura escalable.
2. **Endpoint de Login (Custom)**: *El Gatekeeper*: Se crea un controlador REST personalizado **(ej: /auth/login)** que recibe las credenciales (username y password) en el cuerpo de la petición. Su función es delegar la autenticación al **AuthenticationManager** de Spring, que es la interfaz que maneja la lógica de validación de credenciales.

• Seguridad Básica: JWT y Spring Security (PPT)

1. **Generación del Token:** *La Fábrica de Identidad Digital*: Una vez que el **AuthenticationManager** confirma la identidad del usuario:
   - Se evita la creación de sesión.
   - Un servicio especializado (JwtService) toma los detalles del usuario, añade Claims (carga útil: roles, fecha de expiración, etc.) y usa una clave secreta del servidor para firmar el token, garantizando su integridad.
   - El controlador devuelve el JWT al cliente en el cuerpo JSON de la respuesta.
2. **Acceso a Recursos:** *La Autorización en Cada Petición:* En las peticiones subsiguientes, el cliente debe incluir este JWT en el encabezado **Authorization (Bearer token).** La validación del acceso se realiza sin contacto con la base de datos o sesiones, simplemente verificando la firma del token.

• Implementación en Spring Security

La Cadena de Filtros (Security Filter Chain): El Flujo de Verificación Spring Security funciona a través de una cadena de filtros que procesa secuencialmente cada petición HTTP.

Configuración del Filtro (JwtAuthenticationFilter)

La Identidad en el Contexto: Se añade un filtro personalizado (JwtAuthenticationFilter) en la cadena, usualmente antes del filtro de autenticación por defecto de Spring. Este filtro tiene la función crítica de:

- Extraer el token del encabezado Authorization.
- Validar la firma y la fecha de expiración del JWT.
- Si el token es válido, el filtro crea un objeto Authentication (con la identidad y los roles) y lo carga en el SecurityContextHolder de Spring. Esta acción es la que autentica al usuario para el ciclo de vida de la petición.

• Implementación en Spring Security

Autorización Fina (@PreAuthorize)

Una vez que la identidad y los roles del usuario están cargados en el Contexto de Seguridad, se pueden aplicar reglas de autorización fina a nivel de método.

- Se requiere habilitar esta funcionalidad con @EnableMethodSecurity en la configuración.
- La anotación @PreAuthorize permite definir expresiones complejas para restringir el acceso basado en roles o cualquier otra lógica de seguridad.

• Documentación

Spring Security

JWT

Video de Apoyo

02

Logs estructurados

• Logs Estructurados con SLF4J + Logback

El logging (registro de eventos) es fundamental para la observabilidad de un microservicio.Logs Estructurados: Son registros que se emiten en un formato predecible y fácilmente legible por máquinas, generalmente JSON.

Característica	Descripción
Formato JSON	Permite a las herramientas de agregación de logs (ELK Stack, Grafana Loki) parsear, indexar y buscar información específica rápidamente.
Trazabilidad (Traceability)	Permite inyectar identificadores de traza (trace_id) en todos los logs, facilitando el seguimiento de una solicitud a través de múltiples microservicios.
Consistencia	Asegura que cada entrada de log tenga una estructura uniforme, sin importar el mensaje.

• Logs Estructurados con SLF4J + Logback

Librerías a Implementar

- **SLF4J (Simple Logging Facade for Java):** Es la capa de abstracción. Tu código solo debe interactuar con SLF4J (ej: log.info(...)).
- **Logback:** Es la implementación de logging por defecto en Spring Boot, conocida por su velocidad.
- **Logstash Logback Encoder:** Librería clave que permite a Logback generar la salida en formato JSON, integrando automáticamente campos como la marca de tiempo, nivel y nombre del logger.

¿Por qué usarlos?

Logs = registro histórico de lo que pasa en el sistema:

- Ayudan a detectar errores.
- Permiten reconstruir flujos entre microservicios.
- Son clave para observabilidad y auditoría.

• Logs Estructurados con SLF4J + Logback

SLF4J

Es una API de logging. No escribe los logs, solo define la interfaz.

```java
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Service
public class PedidoService {

    private static final Logger log = LoggerFactory.getLogger(PedidoService.class);

    public Pedido crearPedido(Pedido pedido) {
        log.info("Creando pedido para cliente {}", pedido.getClienteId());
        // lógica...
    }
}
```

Logback

Es el motor de logging por defecto en Spring Boot.Se puede configurar en logback-spring.xml (formatos, niveles, archivos, JSON, etc.).

```xml
<appender name="JSON_FILE" class="ch.qos.logback.core.FileAppender">
    <file>logs/app.json</file>
    <encoder class="net.logstash.logback.encoder.LogstashEncoder" />
</appender>
```

Logs estructurados

“Estructurado” significa que el log tiene un formato consistente para poder ser analizado por herramientas (ELK, Loki, etc.).

```java
log.info("Pedido creado id={} clienteId={}", pedido.getId(), pedido.getClienteId());
```

• Manejo de excepciones globales con @ControllerAdvice

El Manejo de Excepciones Global es la práctica de centralizar la lógica de conversión de excepciones de Java (Exception) en respuestas HTTP semánticamente correctas y predecibles (DTOs de error consistentes).

Librerías y Anotaciones Clave

- **@ControllerAdvice (o @RestControllerAdvice):** Anotación que marca una clase para que intercepte y maneje excepciones lanzadas por cualquier controlador de la aplicación.
- **@ExceptionHandler:** Anotación aplicada a un método dentro de la clase @ControllerAdvice que lo designa como el manejador específico para un tipo de excepción (ej: MiExcepcionPersonalizada.class).

• Manejo de excepciones globales con @ControllerAdvice

Características y Flujo

- **Consistencia de Respuesta:** Garantiza que el cliente siempre reciba una estructura de error (un DTO de error) y un código de estado HTTP apropiado, sin importar qué excepción interna ocurrió.
- **Aislamiento:** Separa la lógica de manejo de errores de la lógica de negocio del controlador, manteniéndolos limpios.
- **Seguridad:** Evita exponer información sensible (como stack traces completos) al usuario final en el caso de un error inesperado.

• Manejo de excepciones globales con @ControllerAdvice

Tipo de Excepción	Status HTTP de Respuesta	Objetivo de la Conversión
MethodArgumentNotValidException	400 Bad Request	Conversión de errores de validación a un JSON legible.
NoSuchElementException (o Custom 404)	404 Not Found	Recurso no encontrado.
Exception.class	500 Internal Server Error	**Manejo de fallos genéricos e inesperados;** ocultando el stack trace.

• Manejo de excepciones globales con @ControllerAdvice

Problema sin ControllerAdvice

- Cada Controller termina repitiendo try/catch.
- Respuestas de error inconsistentes: a veces texto plano, a veces JSON.
- Difícil de mantener.

¿Qué es @ControllerAdvice?

- Es una anotación de Spring que permite definir una clase centralizada para manejar excepciones de todos los controladores.
- Se usa junto con @ExceptionHandler.

• @ControllerAdvice

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(EntityNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(EntityNotFoundException ex) {
        ErrorResponse error = new ErrorResponse();
        error.setMensaje("Recurso no encontrado");
        error.setDetalle(ex.getMessage());
        error.setStatus(HttpStatus.NOT_FOUND.value());
        error.setTimestamp(LocalDateTime.now());

        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }
```

```java
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex) {
        String detalle = ex.getBindingResult().getFieldErrors().stream()
                .map(err -> err.getField() + ": " + err.getDefaultMessage())
                .collect(Collectors.joining(", "));

        ErrorResponse error = new ErrorResponse();
        error.setMensaje("Errores de validación");
        error.setDetalle(detalle);
        error.setStatus(HttpStatus.BAD_REQUEST.value());
        error.setTimestamp(LocalDateTime.now());

        return ResponseEntity.badRequest().body(error);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        ErrorResponse error = new ErrorResponse();
        error.setMensaje("Error interno en el servidor");
        error.setDetalle(ex.getMessage());
        error.setStatus(HttpStatus.INTERNAL_SERVER_ERROR.value());
        error.setTimestamp(LocalDateTime.now());

        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }
}
```

Beneficios

- Formato de error unificado para toda la API.
- Menos código repetido en los controllers.
- Más fácil de documentar (Swagger) y consumir desde frontends.
- Encaja perfecto con ResponseEntity + Bean Validation.

- **Documentación**

Logs estructurados

Video de Apoyo

ControllerAdvice

Video de Apoyo

• ACTIVIDAD PRÁCTICA 1

Usa el proyecto **Proyecto Hospital Vida y Meditación**

Integra manejo de Logs y ControllerAdvice donde creas necesario.

03

¡MUCHAS GRACIAS!

DuocUC®

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS ACREDITADO

Comisión Nacional de Acreditación CNA-Chile

DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.